

Handleiding ANS parametrische figuren maken

Met deze tool kunnen makkelijk parametrische constructies getekend worden in Python.

Stap 1: Bestand opzetten

Download het bestand *plotter.py* op <https://github.com/lisette-924/ANS/blob/main/plotter.py>. Maak vervolgens een leeg *.py*-bestand aan en zorg dat deze in dezelfde folder als *plotter.py* staat. Met het bestand *plotter.py* wordt verder niks meer gedaan.

Kopieer de volgende code naar het lege bestand:

```
from plotter import Structure, Point, PointLoad, Beam, Moment,
Length, DistributedLoad, Support, RotationSpring

from plotter import plot
```

Stap 2: Objecten aanmaken

In deze stap maak je de verschillende onderdelen van je constructie aan. Deze worden in stap 3 toegevoegd aan het figuur.

Zorg dat je als eerste de Points en dan de Beams aanmaakt, vervolgens kan je de andere onderdelen hiermee definiëren.

Punt (`Point`)

Wordt gebruikt om locaties in het vlak aan te duiden.

Argument	Uitleg	Format van input	Standaard waarde
x	Positie van punt op de x-as	Float	-
y	Positie van punt op de y-as	Float	-
label	Label dat in het figuur wordt weergegeven bij het punt	String	''
labelpos	Positie van het label t.o.v. het punt. Kies voor de 1e waarde uit: 'top', 'center' of 'bottom'. Kies voor de 2e waarde uit: 'left', 'center' of 'right'.	(String, String)	('top', 'center')

Voorbeeld:

```
A = Point(0, 0, label='A', labelpos=('left', 'bottom'))
```

Tip: Als je een punt wil definiëren dat bijvoorbeeld 2 meter rechts van A ligt, maar de positie van A is parametrisch, dan kan je dit als volgt doen:

```
B = Point(A.x + 2, 0, label='B', labelpos=('left', 'bottom'))
```

Balk (` Beam `)

Verbindt twee punten.

Argument	Uitleg	Format van input	Standaard waarde
begin	Beginpunt van balk	Point	-
end	Eindpunt van balk	Point	-
label	Label dat in het figuur wordt weergegeven bij de balk. (Kan bijvoorbeeld de stijfheid van de balk zijn.)	String	None
labelpos	Positie van het label t.o.v. het midden van de balk. Kies voor de 1e waarde uit: 'top', 'center' of 'bottom'. Kies voor de 2e waarde uit: 'left', 'center' of 'right'.	(String, String)	('top', 'center')
anglelabel	Kies hier voor True als je in het figuur de hoek van de balk wil aangeven.	Bool	False
anglelabelflip	Kies hier voor True als je het bovenstaande aan de andere kant van de balk wil.	Bool	False

Scharnierende aansluiting toevoegen: Als de balk scharnierend aansluit op de rest van de constructie (en het scharnier dus alleen in de balk zit en niet door meerdere balken), voeg dan `beam.add_hinge(True/False)` toe (True als het scharnier op het beginpunt zit, False als het scharnier op het eindpunt zit),

Voorbeeld:

```
AB = Beam(A, B, label='EI=\u221e', labelpos=('top', 'left'))
```

```
AB.add_hinge(True)
```

Tip: Gebruik `\u221e` voor het oneindig-teken

Steunpunt (`Support`)

Argument	Uitleg	Format van input	Standaard waarde
point	Punt waarop de Support staat	Point	-
support_type	Inklemming = 'fixed' Roloplegging = 'roller' Scharnierende oplegging = 'pinned'	String	'fixed'
angle	Hoek van het steunpunt	float	0.0

Voorbeeld: `As = Support(A, 'roller', angle=-90)`

Tip: Als je een steunpunt wil draaien met dezelfde hoek als een balk (bv. balk AB), gebruik dan `angle=AB.angle`

Puntlast (`PointLoad`)

Argument	Uitleg	Format van input	Standaard waarde
point	Aangrijpingspunt van puntlast	Point	-
value	Waarde van de puntlast. Vul niet in als de waarde nog onbekend is.	Float	None
unit	Eenheid van de kracht	String	'kN'
dx dy	Geef hier de hoek van de kracht aan d.m.v. de verhouding (horizontale verplaatsing, verticale verplaatsing)	(Float, Float)	(0,1)
anglelabel	Kies hier voor True als je in het figuur de hoek van de puntlast wil aangeven.	Bool	False
anglelabelflip	Kies hier voor True als je het bovenstaande aan de andere kant van de puntlast wil.	Bool	False
labelpos	Positie van het label t.o.v. het uiteinde van de pijl. Kies voor de 1e waarde uit: 'top', 'center' of 'bottom'. Kies voor de 2e waarde uit: 'left', 'center' of 'right'.	(String, String)	('top', 'center')
alternative_label	Vul hier iets in als je niet de waarde wil weergeven, maar	String	''

	het bijvoorbeeld een onbekende kracht F is.		
--	---	--	--

Voorbeeld:

```
F = PointLoad(B, value=999, dx dy=(3, -4), anglelabel=True,
alternative_label='F')
```

Moment (`Moment`)

Argument	Uitleg	Format van input	Standaard waarde
point	Aangrijpingspunt van moment	Point	-
value	Waarde van het moment. Vul niet in als de waarde nog onbekend is.	Float	None
unit	Eenheid van het moment	String	'kNm'
clockwise	True als het moment met de klok mee draait in het figuur, anders False	Bool	True
angle	De hoek waarmee je het moment wil draaien zodat de pijl goed zichtbaar is in het figuur.	Float	0.0
labelpos	Positie van het label t.o.v. het aangrijpingspunt. Kies voor de 1e waarde uit: 'top', 'center' of 'bottom'. Kies voor de 2e waarde uit: 'left', 'center' of 'right'.	(String, String)	('top', 'center')
alternative_label	Vul hier iets in als je niet de waarde wil weergeven, maar het bijvoorbeeld een onbekend moment M is.	String	"

Voorbeeld: M = Moment(B, value=5, clock_wise=True)

Verdeelde belasting (`DistributedLoad`)

Argument	Uitleg	Format van input	Standaard waarde
begin_point	Beginpunt van belasting	Point	-
end_point	Eindpunt van belasting	Point	-

begin_value	Waarde van q-last ter plaatse van het beginpunt	Float	-
end_value	Waarde van q-last ter plaatse van eindpunt. Hoeft niet ingevuld te worden als de q-last een constante waarde heeft.	Float	None
unit	Eenheid van de q-last	String	'kN/m'
angle	Hoek van de q-last t.o.v. de x-as	Float	90
n_arrow	Aantal pijltjes	Integer	6
labelpos	Positie van het label t.o.v. begin-aangrijpingspunt. Kies voor de 1e waarde uit: 'top', 'center' of 'bottom'. Kies voor de 2e waarde uit: 'left', 'center' of 'right'.	(String, String)	('top', 'center')
labelpos_end	Positie van het label t.o.v. eind-aangrijpingspunt. Hoeft niet ingevuld te worden als deze hetzelfde is als bij het beginpunt	(String, String)	None
alternative_label_begin	Vul hier iets in als er geen beginwaarde is opgegeven.	String	None
alternative_label_end	Vul hier iets in als er geen eindwaarde is opgegeven. (Niet nodig als label hetzelfde moet zijn als op het begin)	String	None

Tip: wil je de q-last loodrecht op een schuine balk zetten? Gebruik dan `angle=beam.angle +/- 90` (+ of - is afhankelijk aan welke kant je de q-last wilt)

Voorbeeld: `q = DistributedLoad(A, B, begin_value=5, end_value=10, angle=AB.angle-90)`

Rotatieveer (`RotationSpring`)

Argument	Uitleg	Format van input	Standaard waarde
point	Locatie van rotatieveer	Point	-

value	Waarde van de weerstand. Vul niet in als de waarde nog onbekend is.	Float	None
unit	Eenheid van de weerstand	String	'kNm/rad'
labelpos	Positie van het label t.o.v. het midden van de veer. Kies voor de 1e waarde uit: 'top', 'center' of 'bottom'. Kies voor de 2e waarde uit: 'left', 'center' of 'right'.	(String, String)	('top', 'center')
alternative_label	Vul hier iets in als je niet de waarde wil weergeven, maar het bijvoorbeeld een onbekende weerstand r is.	String	"

Voorbeeld: `r = RotationSpring(A, alternative_label='r')`

Maatlijn (`Length`)

Argument	Uitleg	Format van input	Standaard waarde
Point1	Beginpunt van maatlijn	Point	-
Point2	Eindpunt van maatlijn	Point	-
ax	Geef aan langs welke as de maatlijn moet komen ('x' of 'y')	String	'x'
xpos	Als de maatlijn langs de x-as komt, geef dan hier aan of deze boven ('top') of onderaan ('bottom') de figuur moet komen	String	'bottom'
ypos	Als de maatlijn langs de y-as komt, geef dan hier aan of deze links ('left') of rechts ('right') van de figuur moet komen	String	'left'
alternative_label	Vul hier iets in als je niet de afstand wil weergeven, maar dit bijvoorbeeld nog een onbekende lengte x is.	String	None

Voorbeeld: `Lx1 = Length(A, B, ax='x', alternative_label='x')`

Stap 3: Objecten toevoegen aan constructie

Begin met het aanmaken van een constructie: `S = Structure()`

Vervolgens kunnen alle elementen worden toegevoegd. Punten worden automatisch toegevoegd wanneer deze gebruikt is in een onderdeel en hoeven dus niet worden toegevoegd.

Ook scharnieren worden in deze stap toegevoegd met `S.add_hinge(Point)`

De volgende functies kunnen gebruikt worden:

`S.add_point()` -> Deze is eigenlijk overbodig tenzij je ergens een los label wil plaatsen

`S.add_beam()`

`S.add_hinge()`

`S.add_support()`

`S.add_pointload()`

`S.add_moment()`

`S.add_distributedload()`

`S.add_rotationspring()`

`S.add_length()`

Je kan meerdere objecten van dezelfde soort tegelijk toevoegen, bijvoorbeeld:

`S.add_beam(AB, BC, CD)`

Stap 4: Constructie plotten en opslaan

Gebruik `plot(S)` om het figuur te maken en laten zien.

Wil je de afbeelding opslaan?

Gebruik dan:

`plot(S, seed='...')`

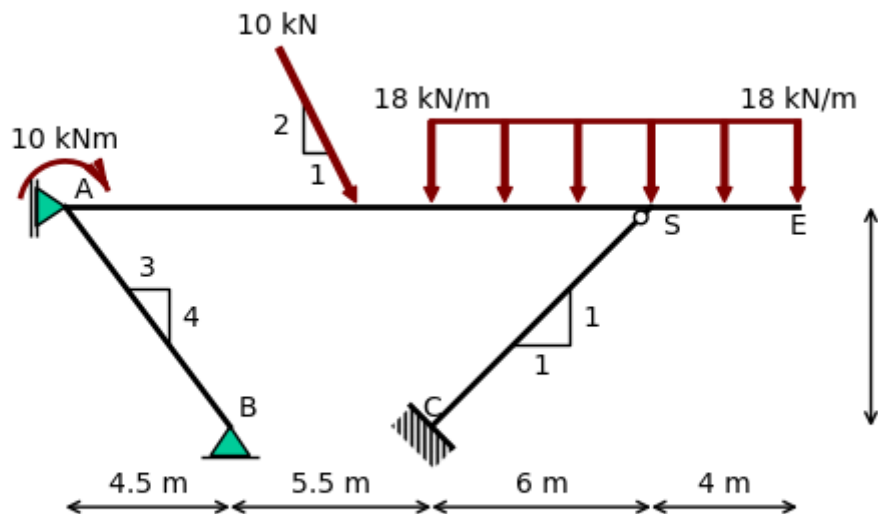
Maak de seed op basis van de gebruikte parameters (bv. `seed='vakwerk{Ax}{Ay}'`).

Vervolgens zal de seed gehasht worden (Er wordt een onherkenbare string van letters en cijfers van gemaakt zodat studenten de afbeelding niet kunnen opzoeken in de database). Wanneer je in ANS het figuur wil toevoegen kan je dezelfde seed gebruiken en hashen. Zolang de input voor de hash hetzelfde is zal de output ook hetzelfde zijn namelijk. Tom weet hier meer over :).

De afbeelding wordt als SVG-format opgeslagen in de folder.

Voorbeeld 1

```
A = Point(0, 6, 'A', ('top', 'right'))
B = Point(4.5, 0, 'B', ('top', 'right'))
C = Point(10, 0, 'C', ('top', 'center'))
D = Point(10, 6, 'D', ('bottom', 'center'))
E = Point(20, 6, 'E', ('bottom', 'center'))
S = Point(16, 6, 'S', ('bottom', 'right'))
AE = Beam(A, E)
AB = Beam(A, B, anglelabel=True)
CS = Beam(C, S, anglelabel=True)
As = Support(A, 'roller', angle=90)
Bs = Support(B, 'pinned')
Cs = Support(C, 'fixed', angle=CS.angle-90)
CS.add_hinge(False)
q = DistributedLoad(D, E, 18)
M = Moment(A, 10, angle=20)
F = PointLoad(Point(8,6), 10, dx dy=(-2, 4), anglelabel=True)
Ly = Length(E, Point(20,0), 'y', ypos='right')
Lx1 = Length(Point(0,0), B)
Lx2 = Length(B,C)
Lx3 = Length(C, Point(16,0))
Lx4 = Length(Point(16,0), Point(20,0))
s2 = Structure()
s2.add_beam(AE, AB, CS)
s2.add_support(As, Bs, Cs)
s2.add_distributedload(q)
s2.add_moment(M)
s2.add_pointload(F)
s2.add_length(Ly, Lx1, Lx2, Lx3, Lx4)
plot(s2)
```

Voorbeeld 2: met parameter Dy

for Dy in [2,3,4]:

```

A = Point(0,0, 'A', ('center', 'left'))
B = Point(4,0, 'B', ('center', 'right'))
C = Point(2,0, 'C', ('bottom', 'center'))
D = Point(2, Dy, 'D', ('bottom', 'right'))
AB = Beam(A, B)
CD = Beam(C, D)
As = Support(A, 'roller')
Bs = Support(B, 'pinned')
Ds = Support(D, 'fixed', angle=180)
q = DistributedLoad(A, B, 2, 13)
M = Moment(C, 10, angle=20+180, labelpos=('bottom', 'center'))
F = PointLoad(B, 10, dxdy=(2, 4), anglelabel=False)
Ly = Length(C, D, 'y', ypos='right')
Lx1 = Length(A, C)

```

```

Lx2 = Length(C, B)
s3 = Structure()
s3.add_beam(AB, CD)
s3.add_support(As, Bs, Ds)
s3.add_distributedload(q)
s3.add_moment(M)
s3.add_pointload(F)
s3.add_length(Ly, Lx1, Lx2)
plot(s3, seed=f'{Dy}')
```

